

django

REST
framework

Part 01

DRF



Introduction

Views

- FBV
- CBV

Responses

Requests

by: Sepehr Bazyar

maktabsharif.ir

What is Django REST Framework?



Introduction



Django REST framework is a powerful and flexible toolkit for building **Web APIs**.

Some reasons you might want to use REST framework:

- The Web browsable API is a huge usability win for your developers.
- Authentication policies including packages for OAuth1a and OAuth2.
- Serialization that supports both ORM and non-ORM data sources.
- Customizable all the way down – just use regular function-based views if you don't need the more powerful features.
- Extensive documentation, and great community support.
- Used and trusted by internationally recognised companies including Mozilla, Red Hat, Heroku, and Eventbrite.

Introduction



Getting Started:

1. Install using ***pip***, including any optional packages you want...

```
pip install djangorestframework
```

2. Add '***rest_framework***' to your ***INSTALLED_APPS*** setting.

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
]
```

What kinds of View are there?



Function Based Views



The `@api_view` decorator for working with function based views.

```
from rest_framework.decorators import api_view

@api_view(['GET', 'POST'])
def post_list_create_view(request):
    """
    List all code posts, or create a new post.
    """
    if request.method == 'GET':
        ...
    elif request.method == 'POST':
        ...
```

Class Based Views



The ***APIView*** class for working with class-based views.

```
from rest_framework.views import APIView

class PostListCreateView(APIView):
    """
    List all code posts, or create a new post.
    """
    def get(self, request):
        ...

    def post(self, request):
        ...
```

How to use Response class?



Responses



REST framework introduces a ***Response*** object, which is a type of ***TemplateResponse*** that takes unrendered content and uses content negotiation to determine the correct content type to return.

```
Response(data, status=None, template_name=None, headers=None, content_type=None)
```

Arguments:

- ***data***: The serialized data for the response.
- ***status***: A status code for the response. Defaults to 200. See also status codes.
- ***template_name***: A template name to use if HTMLRenderer is selected.
- ***headers***: A dictionary of HTTP headers to use in the response.
- ***content_type***: The content type of the response. Typically, this will be set automatically by the renderer as determined by content negotiation, but there may be some cases where you need to specify the content type explicitly.

How to use Request object?



Requests



REST framework's ***Request*** class extends the standard ***HttpRequest***, adding support for REST framework's flexible request parsing and request authentication.

Authentication

REST framework provides flexible, per-request authentication, that gives you the ability to:

- Use different authentication policies for different parts of your API.
- Support the use of multiple authentication policies.
- Provide both user and token information associated with the incoming request.

.user

request.user typically returns an instance of ***django.contrib.auth.models.User***, although the behavior depends on the authentication policy being used.

If the request is unauthenticated the default value of ***request.user*** is an instance of ***django.contrib.auth.models.AnonymousUser***.

Requests



Request parsing

REST framework's Request objects provide flexible request parsing that allows you to treat requests with JSON data or other media types in the same way that you would normally deal with form data.

.data

request.data returns the parsed content of the request body. This is similar to the standard ***request.POST*** and ***request.FILES*** attributes except that:

- It includes all parsed content, including file and non-file inputs.
- It supports parsing the content of HTTP methods other than ***POST***, meaning that you can access the content of ***PUT*** and ***PATCH*** requests.
- It supports REST framework's flexible request parsing, rather than just supporting form data.

.query_params

request.query_params is a more correctly named synonym for ***request.GET***.

Pre-reading



Search about:

- Serializers
- Model Serializers
 - Fields
 - Validations
 - Core Arguments
 - Relationships
 - Customization
 - Overriding Methods
 - ...